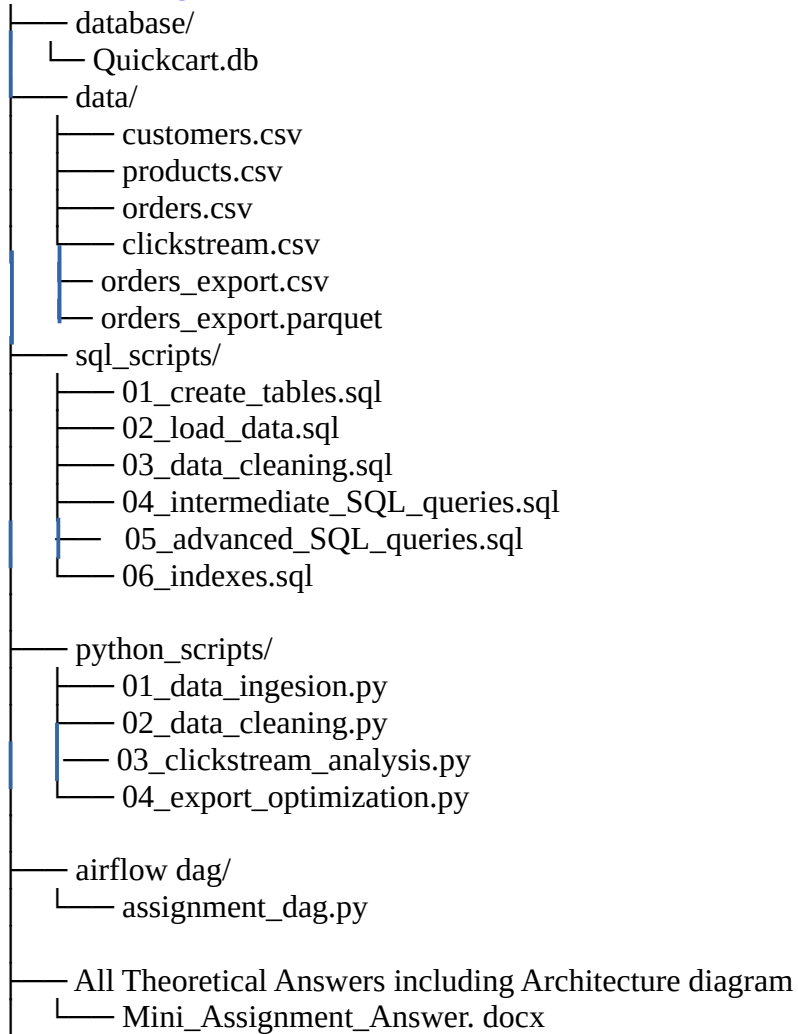


Business case :

Mini_Assignment_Week3/



Section A - Data Engineering Theory

Q1. OLTP vs OLAP Architecture

Answer : 1: The company currently stores transactional and analytical workloads in the same SQL database, it slows down or blocks customers transaction.

Answer 2:

OLTP	OLAP
Online transaction	Online analytical
Real time transaction	Historical transaction
Optimised for speed and consistency	Optimized for aggregation and analysis

Answer 3: Keep OLTP and OLAP database separate, so that OLTP does live transactions and copy them into OLAP for keeping as historical data.

- Answer 4.**
- PostgreSQL /SQLServer/MySQL --- used as database
 - Data warehouse --- used as storage of structured cleaned data
 - Data lake --- used as unstructured data storage

Q2. ACID vs BASE in Distributed Systems

QuickCart wants to store clickstream events from millions of

1. Explain why BASE systems are preferred for clickstream systems.

Ans- Because, clickstreams are high volume and acts better for distributed system.

2. Which workloads require ACID compliance?

Ans – payments, change in order status, updates.

3. Give examples where eventual consistency is acceptable.

Ans – clickstream logs, view count etc.

Q3. Storage Formats & Querying

CSV	JSON	Avro	Parquet
Text format, coma separated, structured	Semi structured, text format,	Binary format,	Binary format, structured
Human readable	human readable	not human readable, Row based, compressed	Human unreadable Column nary based compressed

2. Why do analytical systems prefer columnar storage?

Ans : Columnar storage reads the required columns instead of full rows, so it is much cheaper and faster.

3. Explain avro vs json format.

Ans

JSON

Semi structured, text format,

human readable

Avro

Binary format, faster

not human readable,

Row based, compressed

4. Explain compression benefits in Parquet.

Ans- needs low storage space and cheaper.

Q4. Partitioning, Indexing

Answer:

1. Design a partitioning strategy for:
 - orders table
partition by order date (e.g., monthly)
 - clickstream data
partition by event_date
2. Suggest indexing strategy for:
 - customer lookups
index on customer_id
 - order filtering
order date, payment status
 - joins
use foreign key on customer_id, product_id
3. Explain when indexes negatively affect performance.

Ans A table with frequently writing or updates index acts as negative performance.

SECTION B :

Q3. SQL Optimization Scenario

The following query takes 18 minutes:

```
SELECT c.customer_name,  
SUM(o.quantity * o.unit_price)  
FROM orders o  
JOIN customers c  
ON o.customer_id = c.customer_id  
WHERE order_date >= '2025-01-01'  
GROUP BY c.customer_name;
```

Tasks

1. Identify possible bottlenecks.

Ans : The query scans the full orders table, there is no indexing. Database should examine row wise to find where order date is '2025-01-01'.

There is sum of many rows, it is problematic if the order table contains huge no of orders.

2. Suggest indexes.

Ans Sql scripts for indexing- index on customer_id, order_date and composite index on customer_id and order_date.

3. Explain execution plan analysis.

Ans: Execution plan shows whether indexes are used.

The order in which tables are accessed.

Ans : Whether temporary structures are created for sorting groups.

4. Suggest partitioning improvements.

Ans Partition over order_year or order_month can be used.

5. Explain materialized views usefulness.

Materialized view stores the results of query rather than calculating them. For example calculating customers revenue for every one one can create customer wise revenue.

Q4. Database Design

Design schema for: 1. Shipment tracking 2. Product inventory 3. Customer support tickets

Requirements: • normalized design • primary keys • foreign keys • indexing recommendations

Ans :

1. Shipment tracking -----> shipment_id(PK), order_id, order_date, delivery_date, shipment_status, FK: order_id, References orders(order_id)

2. Product inventory -----> inventory_id(PK), product_id, FK: product_id, References products(product_id)

3. Customer Support Tickets -----> ticket_id (PK), customer_id reference (customers), order_id (FK) reference order

Each schema follows 3NF

Section D - ETL/ELT + Airflow

Q1. ETL vs ELT Architecture

1. Compare ETL vs ELT.

Ans

ETL

Follow the order: Extract, transform then load
Difficult to reprocess
Need clean data before loading
Clean data before a restricted destination

ELT

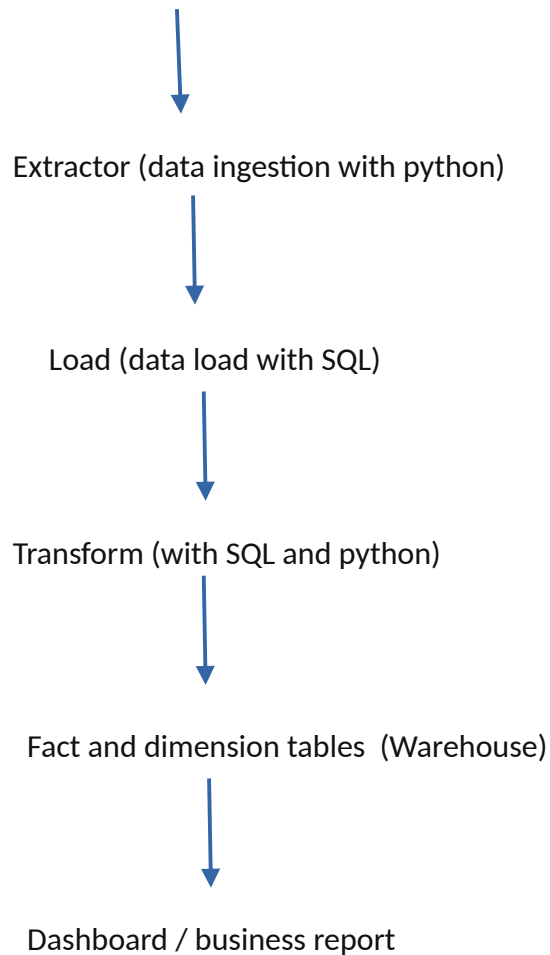
Follow the order: Extracts, load, transform
Easier to reprocess
Any type of data can be loaded
Keep raw data and transform in a powerful warehouse

2. Explain why cloud warehouses prefer ELT.

Ans: Warehouse prefer ETL to ensure data quality, security and resource control

3. Design modern ELT pipeline for QuickCart.

Ans: CSV files (customers, orders, products, clickstreams)

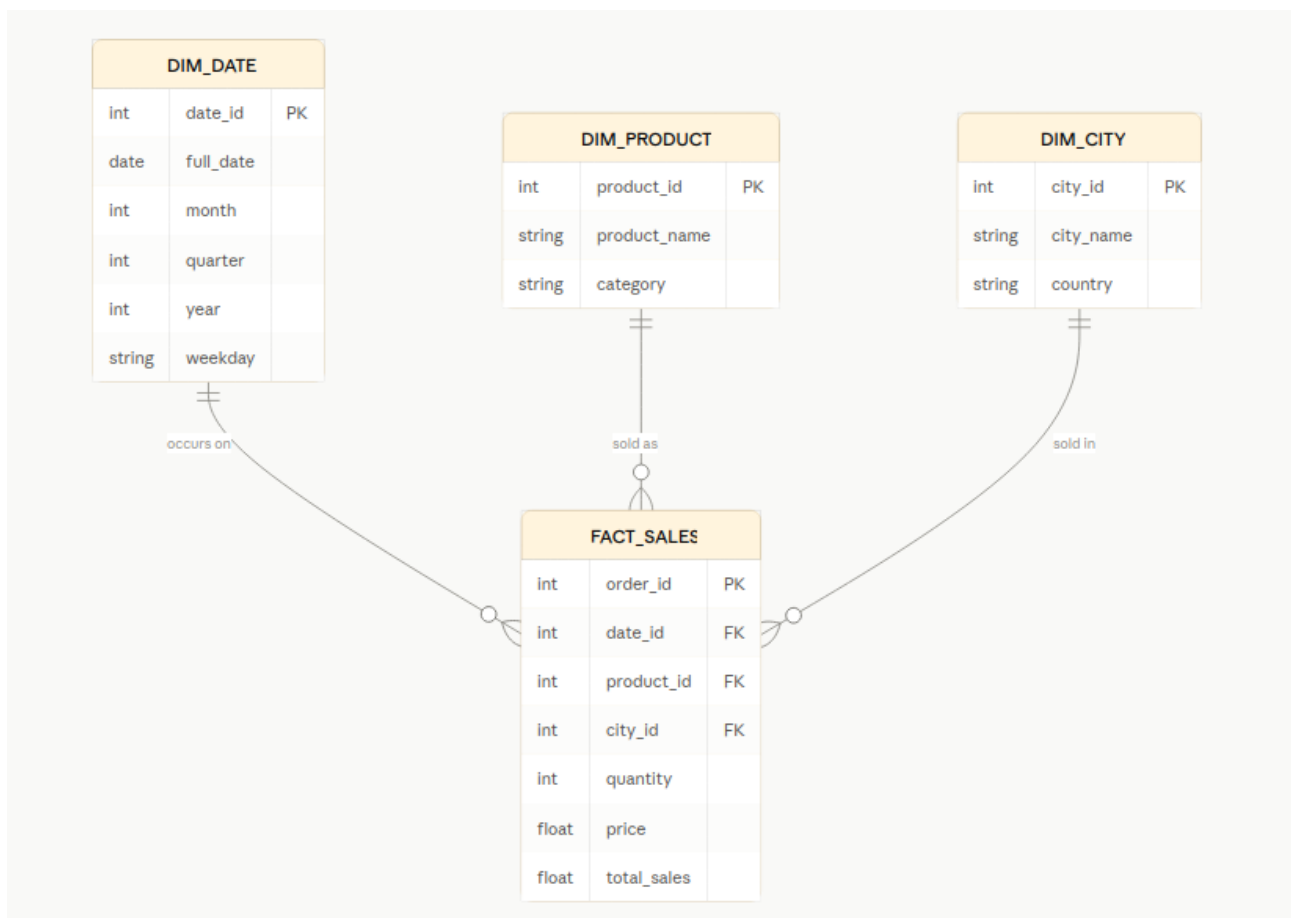


Q3. Data Warehouse Modeling

Tasks

1. Design star schema.
2. Identify:
 - fact tables
 - dimension tables

Ans : Star Schema :



Fact table is Fact_Sales: this is the center of the star schema

Dimension tables are: Dim_date, dim_product, dim_city

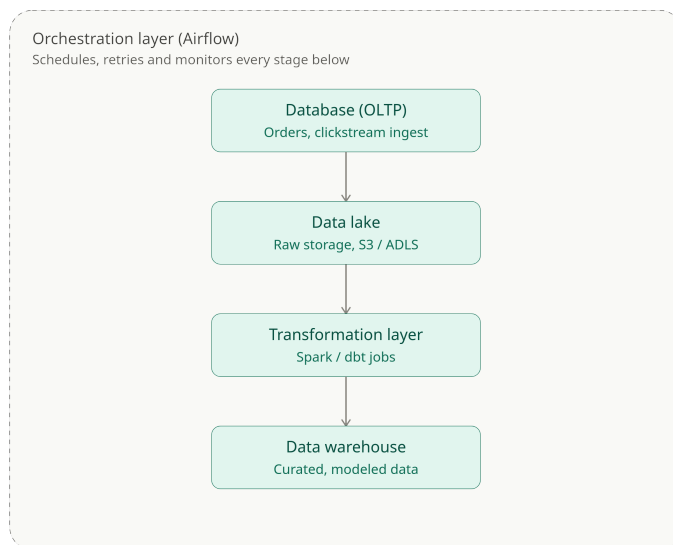
Section E - Performance Engineering Case Study

Part 1 - Architecture Design

Design scalable architecture using:

- database
- data lake
- warehouse
- orchestration
- transformation layer

Ans



1. Database (OLTP)

The main database, where new orders, customers information are stored. This should be able to handle millions of data or records everyday, it can use multiple servers instead of one.

2. Data Lake

The data lake stores all the raw data exactly as it comes from the database.

3. Transformation Layer

This step cleans the raw data. It removes duplicate records, fixes errors, combines data from different sources, and prepares it for analysis.

4. Data Warehouse

The cleaned data is stored in a data warehouse. This is used for dashboards, and business analysis because it allows queries to run quickly.

5. Orchestration

Airflow connects the whole workflow. It starts each task in the correct order, checks whether tasks finish successfully, retries failed tasks, and sends alerts if something goes wrong.

Part 2 - Optimization

Explain:

1. Partitioning strategy
2. Indexing strategy
3. Compression strategy
4. Query optimization
5. Incremental loading
6. Columnar storage benefits

1. Partitioning

Divide the data into smaller parts, could be by date. This allows queries to read only the required item instead of the entire dataset, making them much faster.

2. Indexing

Create indexes on important columns like **order_id**, **customer_id**, and **date**. This helps the database to find data more quickly.

3. Compression

Compress the data before storing it. This reduces storage space and can make performance faster.

4. Query Optimization

- Select only the columns according to requirement.
- Filter the data as early as possible.
- Store frequently used summaries so they do not need to be calculated every time.
- Avoid unnecessary joins.

5. Incremental Loading

Instead of processing all the data every day, process only the new or updated records. This makes the pipeline faster and reduces failures.

6. Columnar Storage

Store data by columns instead of rows. This makes analytical queries much faster because only the needed columns are read.

Part 3 - Reliability

Explain:

1. Retry handling
2. Failure alerts
3. Data quality validation

1. Retry Handling

If a task fails because of a temporary problem, Airflow automatically retries it. This helps the pipeline complete successfully without manual work.

2. Failure Alerts

If a task still fails after retries, Airflow sends an alert via e-mail, so that assigned part team can fix the problem quickly.

3. Data Quality Validation

Check the data before loading it into the warehouse.

Examples:

- Check that the expected number of rows is received.
- Check for missing or duplicate data.
- Verify that all required columns are present.
- Make sure the data is recent and not outdated.

This ensures that the reports and dashboards use correct and reliable data.